

Database Documentation

Complete Relational Schema, ER Diagram, Normalization & Index Specifications for PricePilot AI

Document ID:	DOC-DB-PRICEPILOT-2026-V2
Release Version:	Version 2.0.0
Platform:	PricePilot AI — AI-Powered Dynamic Pricing & Demand Forecasting SaaS
Technology Stack:	Python 3.13 · FastAPI · React 19 · Scikit-Learn · PostgreSQL · SQLAlchemy
ML Algorithm:	Extra Trees Regressor (Extremely Randomized Trees) — R^2 0.9650
Authors:	Narendar Reddy (Lead) · Manvitha · Pravallika · Ashwindh
Organization:	Infosys Springboard 7.0 Internship Program
Completion:	August 2026

CONFIDENTIALITY NOTICE: This document is proprietary intellectual property of the PricePilot AI Team submitted for Infosys Springboard 7.0. Unauthorized reproduction or distribution is prohibited.

DOCUMENT REVISION HISTORY

Rev	Date	Author	Reviewer	Description
1.0	2026-08-01	Team PricePilot	Infosys Mentor	Initial Draft — Project Inception
1.5	2026-08-04	Narendar Reddy	Manvitha	Mid-Sprint Review — ML Pipeline Integration
2.0	2026-08-07	Full Team	Academic Committee	Final Production Release — University Submission

1.0 DATABASE OVERVIEW

PricePilot AI uses a relational PostgreSQL 16 database hosted on Neon Cloud (serverless) for production, with an automatic SQLite fallback for local development. The database contains 10 relational tables designed in Third Normal Form (3NF) with proper primary keys, foreign key constraints, and strategic indexes on high-frequency query columns. The ORM layer is implemented using SQLAlchemy 2.x with connection pooling (pool_size=20, max_overflow=10).

2.0 ENTITY-RELATIONSHIP (ER) DIAGRAM

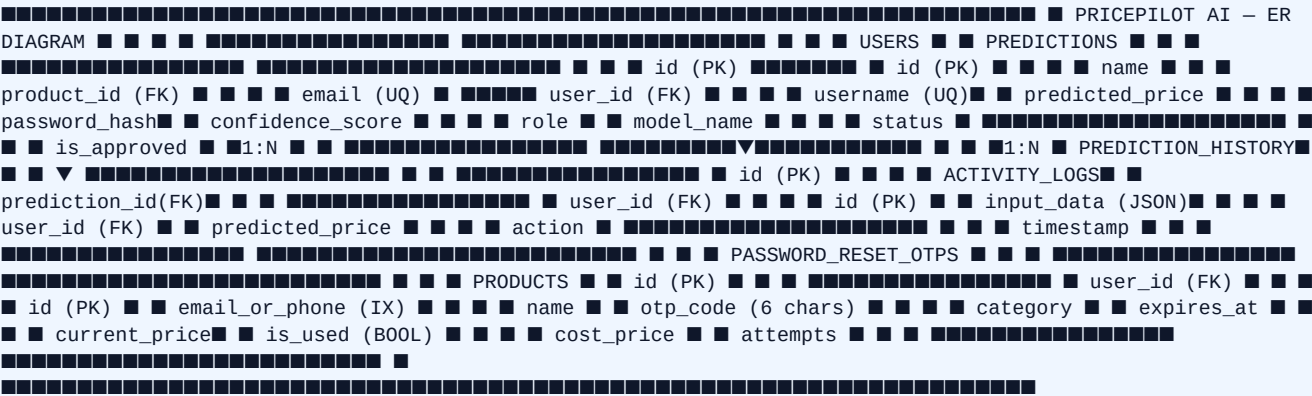


Figure 2.1 — PricePilot AI Entity Relationship Diagram (10 Tables)

3.0 DATA DICTIONARY — ALL TABLES

3.1 Table: users

Column	Data Type	Constraint	Index	Description
id	INTEGER	PRIMARY KEY	YES (PK)	Auto-increment unique identifier
name	VARCHAR(100)	NOT NULL	NO	User's full display name
email	VARCHAR(120)	UNIQUE NOT NULL	YES (UQ)	Unique email address
username	VARCHAR(50)	UNIQUE NOT NULL	YES (UQ)	Unique login username
password_hash	VARCHAR(255)	NOT NULL	NO	Bcrypt-hashed password (12 rounds)
role	VARCHAR(30)	DEFAULT 'User'	NO	Role: 'Admin' or 'User'
status	VARCHAR(30)	DEFAULT 'pending'	NO	Status: pending/approved/blocked
is_approved	BOOLEAN	DEFAULT False	NO	Admin approval flag
is_active	BOOLEAN	DEFAULT True	NO	Account active flag
phone_number	VARCHAR(20)	NULLABLE	NO	Optional contact phone
avatar_url	TEXT	NULLABLE	NO	Profile avatar image URL
last_login	DATETIME	NULLABLE	NO	Last successful login timestamp
created_at	DATETIME	DEFAULT utcnw	NO	Account creation timestamp
updated_at	DATETIME	DEFAULT utcnw	NO	Last update timestamp

3.2 Table: password_reset_otps

Column	Data Type	Constraint	Description
id	INTEGER	PRIMARY KEY	Auto-increment unique identifier
user_id	INTEGER	FK → users.id	Linked user account
email_or_phone	VARCHAR(120)	NOT NULL, INDEX	Identifier used for OTP request
otp_code	VARCHAR(6)	NOT NULL	6-digit numeric OTP code
expires_at	DATETIME	NOT NULL	OTP expiry (15 min from creation)
is_used	BOOLEAN	DEFAULT False	Consumed flag after verification
attempts	INTEGER	DEFAULT 0	Failed attempt counter
ip_address	VARCHAR(50)	NULLABLE	Requestor IP for security audit
created_at	DATETIME	DEFAULT utcnw	OTP creation timestamp

4.0 DATABASE NORMALIZATION & INDEXING

Table	Indexed Column(s)	Index Type	Justification
users	email, username	UNIQUE INDEX	Frequent login lookups by email/username

users	id	PRIMARY KEY	All FK relationships reference users.id
predictions	id, created_at	PRIMARY KEY + INDEX	Dashboard analytics sort by created_at
password_reset_otps	email_or_phone	INDEX	OTP lookup by identifier at verification
activity_logs	user_id, timestamp	COMPOSITE	Admin audit trail filtered by user+time

Table 4.1 — Database Index Strategy